ST215: direct transfer through contextual reassociation

A materially different 215-state consistency machine

Research contribution for Vladimir Reshetnikov's ProveIt project
Prepared with AI-assisted construction and independent computational and mathematical review
21 October 2026

Abstract

We give a deterministic, one-tape, two-symbol machine with 215 working states whose blank-tape nonhalting is equivalent to Con(ZFC), by an ordinary proof formalizable in ZFC and using the repository's established numerical-calculus calibration. The new implementation realizes a direct, 35-leaf transfer command previously blocked by two physical source-context constraints. Reassociating the outer source tree satisfies both constraints without changing any edge of its complete ordinal control-flow graph. The source has 201 distinct subtrees and 4,732 primitive occurrences. All 6,151 universal whole-tape primitive cases pass; every one of the 430 literal transition rows is covered. An exact 70,506-step blank run matches the interior of an axiom command after a 177-step source prefix. Completing that command places ten theorem heads above an opaque stack tail; the scheduler requires only two heads of protected credit. The new table is distinct, up to rooted state relabeling and reflection, from the compared 215-state tables in the pinned repository and twelve incoming archives. The 214-state target is not reached. We also give a command-language obstruction for rotating pop, correct a false return edge in an earlier scratch-biased experiment, and prove an abstract adjacent-candidate synchronization lemma that may support future encodings. The archive contains exact tables, editable TeX, the PDF, proofs, deterministic reconstruction, complete startup data, and offline certificate replay.

Contents

1 Result, conventions, and novelty **2**

2 Numerical foundation: no formula-decoding requirement **3**

3 Three commands and the direct transfer **4**

4 The contextual reassociation that makes it fit **4**

5 Whole-tape physical simulation **5**

6 Exact blank startup and the partial AX bridge **6**

7 A complete strong compiler for the direct command **7**

8 Protected credit for the actual scheduler **9**

9 The consistency equivalence **10**

10 Other formalisms and research results **11**

11 Physical comparison and the remaining 214-state problem **12**

12 Reproduction, artifacts, and inherited boundaries **13**

A Independent ST215 bootstrap and calibration review **15**

B A productive-adjacency obstruction for rotating pop **18**

C A control-flow error in the archived scratch-biased experiment **20**

D Consecutive successful codes for an abstract biased parser **21**

E Complete dense transition table **23**

1 Result, conventions, and novelty

The deliverable is the new machine **ST215**. It has 215 working states and a separate halting label, with 430 total transition rows. The native table starts in state 0, uses dispatcher 84, and halts on entering 254. The dense presentation uses working states $0, \ldots, 214$, start 0, and halt 215. Both start with the head at cell 0 of a bi-infinite all-zero tape; every transition writes one bit and moves one cell left or right. No initial program or data are supplied externally.

Theorem 1 (Main result)**.** *Using the pinned inherited numerical-calculus calibration and all-natural arithmetic macro theorems, the construction and proofs in this packet establish*

$$\mathrm{ZFC} \vdash \big(\neg\,\mathrm{Halt}(\mathrm{ST215}, \mathrm{blank}) \longleftrightarrow \mathrm{Con}(\mathrm{ZFC})\big).$$

This is an ordinary mathematical result with replayable symbolic and finite certificates. We supply no newly compiled exact-table Lean or Rocq endpoint. The published K215 proof has the same ordinary-versus-kernel distinction at the pinned snapshot [1, 2]. The foundational calibration is reused, not replaced by a newly invented consistency assumption.

Quantity	Published K215	Composite parent	ST215
Working states	215	215	215
Distinct source subtrees	202	201	201
Primitive occurrences	4,751	4,775	4,732
Guarded primitive cases	6,176	6,209	6,151
Transfer body occurrences	—	78	35
Symbolic inference steps	1,698,790	1,611,489	1,617,630
Minimum duration constant	6	7	5
Blank steps to certified frame	95,714	68,442	70,506

The transfer parent is the incoming T215 packet's frozen `reference/transfer-parent`, not the published K215 machine [4]. Smaller source size does not automatically mean fewer physical states: tree dispatch, arithmetic helpers, register scans, and initialization share states under context-sensitive contracts. Nor does the smaller primitive count prove faster execution.

The material change has three parts. First, the command language has only transfer, modus ponens, and axiom generation, with field arities $1, 2, 3$. Second, transfer is now the direct 35-leaf implementation testing only the incoming cache, in place of the 78-leaf composite testing both top and cache. Third, the source is placed in a new outer tree context, with a zero-valued spare-block refill and a different calibrated blank startup. It is not a state renaming of an existing compared table.

The native table's SHA-256 is

`d5237504325eee96b0040d4d9574731a1caf4bcda3d6a49e94827d9532a664e1`

The dense table's SHA-256 is

`ead0ec3aa1c3502015faa8a077ded98be7522fa907a9df43a8052de555398375`

The source's SHA-256 is

`212cf0ffde43c1cc22c183dd8bf0b3a699396f96c9942ee92a7cc372cd273ca9`

2 Numerical foundation: no formula-decoding requirement

For natural numbers put

$$\mathsf{P}(a,b)=\frac{(a+b)(a+b+1)}{2}+b,\qquad \mathsf{I}(a,b)=4\mathsf{P}(a,b),\qquad \mathsf{Q}_a b=4\mathsf{P}(a,b)+2.$$

Write $a\to b=\mathsf{I}(a,b)$, falsity $F=1$, truth 0, and $NX=\mathsf{I}(X,1)$. Also put $M(a,b)=4\mathsf{P}(a,b)+1$, $E(a,b)=4\mathsf{P}(a,b)+3$, and $\exists_a X=N\mathsf{Q}_a NX$. Parameters range over *all natural numbers*. No formula parser, variable-support test, or well-formedness filter is inserted. In particular $\mathsf{I}(0,0)=0$, and unpair$(0)=(0,0)$. Those aliases are used as literal numerical equalities.

The retained calculus $\mathcal{D}$ has truth, numerical MP, theorem-only GEN, seven logical rows, the original gated bundle, and the dominated row below:

$$\begin{aligned}
R_1(a,b)&=\exists_a E(a,b),\\
R_2(a,b,c)&=\mathsf{Q}_a(b\to c)\to(\mathsf{Q}_a b\to\mathsf{Q}_a c),\\
R_3(a,b,c)&=E(a,b)\to(M(c,a)\to M(c,b)),\\
R_4(a,b,c)&=\mathsf{Q}_a\mathsf{Q}_b c\to\mathsf{Q}_b\mathsf{Q}_a c,\\
R_5(a,b,c)&=E(c,a)\to(M(c,b)\to M(a,b)),\\
R_6(a,b,c)&=(\exists_a\mathsf{Q}_a c)\to c,\\
R_7(a,b,c)&=E(c,a)\to(E(c,b)\to\mathsf{Q}_{M(a,b)}E(a,b)),\\
L(A,B,C,U)&=((C\to U)\to A)\to((A\to C)\to(B\to C)),\qquad U\le A.
\end{aligned}$$

The unused b in R_6 is intentional and agrees with the literal source. The enabled bundle is retained in full:

$$\begin{aligned}
B_0(b,k,m)&=\mathsf{Q}_b(m\to N\mathsf{Q}_b(m\to NM(b,k))),\\
S_0(a,b,k)&=\mathsf{Q}_k(M(k,a)\to M(k,b)),\\
C_0(a,b,k,m)&=\mathsf{Q}_b(M(b,k)\to N\mathsf{Q}_a(m\to N\mathsf{Q}_a(S_0(a,b,k)\to B_0(b,k,m)))),\\
J_0(a,b,c)&=\mathsf{Q}_c(C_0(a,b,c,c)\to NB_0(b,c,c)),\\
V_0(a,b,c)&=M(a,b)\to\mathsf{Q}_c M(a,b),\\
\mathrm{Bundle}(a,b,c)&=N(V_0(a,b,c)\to J_0(a,b,c)).
\end{aligned}$$

It is an axiom when $c\neq a,b$. The gate outputs truth otherwise. Removing a support-bearing part of this bundle would require a new calibration; we do not do so.

The inherited finite recovery certificate proves that the dominated calculus has the same natural-code theorem set as its unrestricted predecessor. It contains 3,728 proof nodes and 5,160 shared term nodes. The included checker verifies explicit substitutions, MP, bijective renamings, and exact endpoints. A dominated leaf is justified because its U is zero or a literal subterm of A; numerical implication dominates its arguments, so every natural substitution preserves $U\le A$. The only term normalization is $\mathsf{I}(0,0)=0$. Universal schema instantiation in this finite metaproof does not silently add a substitution inference rule to the numerical machine.

The further inherited calibration identifies numerical refutation with ZFC refutation:

$$\mathcal{D}\vdash 1\quad\longleftrightarrow\quad\exists p\ \mathrm{Proof}_{\mathrm{ZFC}}(p,\bot).\tag{1}$$

This is the established foundation of the repository's ordinary K215 proof [2, 3]. It is stated here as a reused theorem, not re-proved from first-order syntax. The distinction matters: checking the 3,728-node propositional certificate alone does not establish the whole comparison with ZFC.

Thus the user-permitted freedom to use junk codes is already present in the actual formalism. The advance in this packet comes from a smaller operational presentation and its physical implementation, rather than from a newly claimed axiomatization of set theory.

3 Three commands and the direct transfer

Write a command store as (t, w, d, h): top, encoded stack, additive counter, and cached formula. The notation $[x; w] = \mathsf{P}(x, w)$ extends by right nesting to a finite explicit list over any numerical tail. For $\mathrm{unpair}(w) = (x, v)$ the new command is

$$\mathsf{S}(t, w, d, h) = \begin{cases} \mathsf{halt}, & h = 1, \\ (h, v, d, x), & h \neq 1. \end{cases} \tag{2}$$

An incoming top $t = 1$ alone does not halt. The predecessor composite DROP; NIP; NIP had the additional guard $t = 1$. Both forms have the same nonhalting store map, but the single-guard contract is the exact one for which the original three-command strong compiler was proved.

For $y = d + x$, modus ponens is

$$\mathsf{MP}(x)(t, w, d, h) = \begin{cases} (y, [0; w], y, 0), & \mathsf{I}(t, y) = h, \\ (0, w, y, 0), & \mathsf{I}(t, y) \neq h. \end{cases}$$

For $\mathsf{AX}(c; x, y)$ put $U = d$, $A = d + x$, $B = h + y$, $C = c$. Its output is

$$\big(G(A, B, c), [R_1, \ldots, R_7, L(A, B, c, U), \mathsf{Q}_U h, t; w], 0, 0\big), \tag{3}$$

where $R_j = R_j(A, B, c)$ and G is the full bundle or zero according to its gate. Here h in $\mathsf{Q}_U h$ is the incoming cache, before adding y. The seven rows, L, and the gate are implication-coded, and hence not 1; $\mathsf{Q}_U h$ has tag 2 and is neither 0 nor 1. The tenth head saves the incoming top.

The field lists are exactly

$$f(\mathsf{S}) = [0], \qquad f(\mathsf{MP}(x)) = [1, x], \qquad f(\mathsf{AX}(c; x, y)) = [c + 2, y, x].$$

The source's logical registers, in order, are $(N, T, W, A, D, H, X, Y, L)$, with N the next candidate, A the selector, X, Y scratches, and L the residual command pointer. At a complete command boundary $A = X = Y = 0$.

Why the direct body is only 35 occurrences. At source ordinal 4697, clear T in a two-leaf loop. Next, while decrementing H, increment both T and X; move X back into H. This seven-leaf copy leaves $T = H = h$ and $X = 0$. Apply the unchanged 26-leaf NIP. That body tests $H = 1$; otherwise it clears H, unpairs W, leaves the tail in W, and loads the head into H. Thus the total is $2 + 7 + 26 = 35$, and the map is (2). All copying loops decrease a natural register. The inherited NIP theorem handles its guard and unpairing on every natural input, including $W = 0$.

The direct body preserves N, L, D and returns zero A, X, Y on a nonhalting exit. Its only halt is primitive CH at ordinal 4717. The replacement saves 43 occurrences relative to the 78-leaf composite, while maintaining 201 distinct source subtrees.

4 The contextual reassociation that makes it fit

The incoming short-transfer experiments already identified the 35-leaf semantic body. A straightforward placement failed its physical CD context; a subsequent context-root placement fixed CD but failed its return continuation. The present construction repairs the *outer source context*, without an additional physical helper or a modified candidate scheduler.

Write the context-root tree as

$$t = (A, ((B, C), D)),$$

where $A = (\text{scheduler}, \text{field reader})$, B contains the selector and AX/MP bodies, C clears top, and D copies cache then performs NIP. Set

$$t' = (((A, B), C), D). \tag{4}$$

This is not an arbitrary associativity law for a structured-control language: ancestor-relative breaks and continues can change their targets when the tree changes. Here preservation is checked for *every* guarded source edge by two independently implemented resolvers.

Purpose	Ordinal	Old address	New address
Return to scheduler, CC3	4696	10011	00111
Clear top, CD1	4697	1010	010
Unique halt, CH	4717	new address 11101	

CC3 still chooses its third right ancestor, which is the root. Its new address ends in 111, the suffix required by the existing compressed physical continuation helper. CD1 has the required preceding marker bit in its new context. These local observations explain the repair; the universal whole-tape runtime certificate is what establishes the physical implementation.

All 4,732 source addresses change relative to context-root, while the ordered opcode/operand list is identical. The two resolvers agree on all 6,151 guarded cases. The complete ordinal CFG also equals that of the incoming simple direct-transfer source. Only the syntactic spelling of its return differs there; the target is unchanged. The tree is closed, has 201 distinct subtrees, 4,731 internal prefix occurrences, maximum depth 44, and maximum zero run 6.

The deterministic reconstruction begins with the frozen transfer parent source, extracts its seven-leaf cache copy and 26-leaf NIP, forms the direct body, makes the context-root transformation, and then applies (4). The native table is reconstructed from the parent's literal table by an explicit row patch. The noteworthy final row

$$198 \quad 0 \quad 0 \quad L \quad 40$$

makes newly appended spare blocks zero-valued. The source semantics are independent of spare values, but initialization is not; the blank proof below concerns this exact row and start state 0. The full recipe is executable in `checks/rebuild.py`.

5 Whole-tape physical simulation

The certificate covers every primitive occurrence, with separate zero and positive cases for each decrement. It is not a collection of bounded integer tests. Each case quantifies over all natural primary register values, all nonnegative gaps, and every nonempty finite list of spare values.

A standard frame, with dispatcher head at position p, has a left marker at $p - 1$, the finite source-address bits beginning at p, a zero gap to a header marker at $p + 64 + g$, and the register file immediately to its right. Register blocks are

$$B(v) = 0\,1^{v+1}.$$

Logical registers map to physical block indices by

$$(6, 2, 0, 4, 3, 7, 8, 5, 1).$$

After the nine primary blocks comes a nonempty arbitrary finite spare list; all remaining tape is zero. The certificate proves the complete tape relation, not only selected registers.

The proof engine represents run lengths by affine expressions with 13 coordinates: a constant coordinate, gap, nine primary values, first-spare value, and the width of the remaining spare list. All variable coordinates denote nonnegative values. It uses three sound steps:

1. A literal step checks a row of the exact table and rewrites one cell, with exact head motion and elapsed time.
2. A unary step proves a directional self-loop over a homogeneous run by induction on its arbitrary natural length.
3. A list step applies a separately derived one-block schema by structural induction over an arbitrary finite list of blocks.

The four one-block schemas have duration $v + 2$. They scan B right, convert B to $C = 1\,0\,1^v$ right, scan B left, or convert C back to B left. The list normalization $C(A)\,1 = 1\,B(A)$ follows by induction on the block list A. Opaque left and right contexts in the one-block proofs ensure that no unexamined tape cell is changed.

Every symbolic case is derived from the literal table after an explicit injective relabeling to the engine's historical labels. An increment appends a zero spare. No primitive deletes the final spare; the represented frame class is therefore closed. The output address and gap changes agree with source successor, break, continue, increment, and decrement semantics. The unique CH case reaches the external halting label.

Exact-table obligation	Observed result
Guarded cases proved	6,151 of 6,151
Literal table rows covered	430 of 430
Symbolic inference steps	1,617,630
Smallest duration constant	5
Failed cases	0
Independent block schemas	4

Each duration has nonnegative coefficients and a positive constant. Thus every source step takes a finite positive number of binary steps, including the halting case. Infinite source execution cannot collapse to a finite physical execution. A finite complete source run lifts to a finite binary run.

The saved receipts are `evidence/runtime.json` and `evidence/runtime_cases.json.gz`. The default replay audits their bindings, order, coverage, durations, and four block schemas. The explicit `-runtime` option rederives all 6,151 cases and compares every resulting case record with the saved record. Python and the ordinary soundness argument are part of this verification boundary; these are not proof-assistant kernel certificates.

6 Exact blank startup and the partial AX bridge

Literal execution from state 0 on blank tape reaches the following complete frame after exactly **70,506 binary transitions**:

$$q = 84, \qquad p = -2, \qquad \text{header} = 62, \qquad g = 0,$$
$$\text{PC} = 00110010010110010, \qquad \text{ordinal} = 58,$$
$$(N, T, W, A, D, H, X, Y, L) = (1, 0, 0, 0, 0, 0, 0, 18, 0).$$

There are 63 spare blocks, every one zero-valued, and exactly 100 one-cells on the entire tape. The full one-cell set is recorded in `evidence/bootstrap.json`; every other cell is zero. The independent native and dense executions agree at every step. `evidence/blank-delta.jsonl.gz` contains all 70,506 transition deltas, not merely sampled snapshots.

This is an interior AX frame, not a clean source-root configuration. To identify it, execute the source from AX entry ordinal 25 with

$$(N, T, W, A, D, H, X, Y, L) = (1, 0, 2, 0, 0, 0, 0, 0, 0).$$

After exactly 177 primitives it reaches ordinal 58 and the complete displayed logical store. Both CFG resolvers produce the same trace. The full before/after stores, literal instruction, and next ordinal are stored for every source transition in `evidence/AX-prefix.json`. An additional independent checker constructs the physical frame directly from the native table and the register-block definition.

The inherited all-natural AX theorem now applies to the *remaining suffix* of this exact command. Its synthetic input is $\mathsf{AX}(0;0,0)$ on $(t,w,d,h)=(0,2,0,0)$. Completion terminates and yields

$$N=1,\quad L=0,\quad A=X=Y=0,\quad t=d=h=0,$$
$$w=[R_1(0,0,0),\dots,R_7(0,0,0),L(0,0,0,0),\mathsf{Q}_0 0,0;\,2].$$

All ten explicit heads are native theorems: seven retained axiom instances, the dominated row, a generalization of truth, and truth. The gate is zero because $c=A=B=0$.

The suffix 2 is deliberately treated as opaque. Indeed $2=\mathsf{P}(0,1)$ and $1=\mathsf{P}(1,0)$, so blindly popping it would eventually expose falsity. The credit theorem in Section 8 prevents this. Merely reaching a plausible register frame, or calling the tail harmless, would not establish soundness.

It is unnecessary to literally simulate the enormous formula codes generated by the rest of AX. Exact matching of a finite prefix, the all-natural terminating AX theorem, and positive-duration primitive simulation determine its future completion. No asymptotic or probabilistic startup assumption is used.

7 A complete strong compiler for the direct command

For completeness, the compiler must work over *arbitrary* retained top and stack, not only theorems. For every $\mathcal{D}$-theorem X, construct a finite command block B_X such that

$$(t,w,0,0)\xrightarrow{B_X}\mathsf{halt}\quad\text{or}\quad(X,w,0,0)\qquad\text{for all natural } t,w.\qquad\text{(Strong)}$$

Early halting counts as success. Soundness of actual runs is a separate invariant. The inherited compiler [4] was proved for exactly the single-guard contract (2); there is no need to infer it backwards from the more eager composite guard.

7.1 Structural macros

Put $D=\mathsf{AX}(0;0,0)$ and

$$H=D;\mathsf{S}^{11},\qquad \mathrm{PUSH}=D;\mathsf{S}^{9};\mathsf{MP}(0).$$

After ten transfers following D, the tuple is $(\mathsf{Q}_d h,w,0,t)$. None of the first ten tests sees a new falsity head: only zero and the first nine generated entries have been tested. The eleventh transfer halts precisely if $t=1$; otherwise, writing $w=[x;v]$, it yields $(t,v,0,x)$. Thus H preserves top, pops one preexisting head, resets the counter, and tests the *incoming top*. It is not a substitute for the old NIP's incoming-cache test.

After nine transfers in PUSH the tuple is $(L,[t;w],0,\mathsf{Q}_d h)$. Its final MP necessarily fails because the cache has tag 2 whereas $\mathsf{I}(L,0)$ has tag 0. Hence

$$\mathrm{PUSH}(t,w,d,h)=(0,[t;w],0,0)$$

and PUSH never halts, even when the original t or h equals 1.

Define

$$\mathrm{DISCARD}=\mathsf{MP}(1);\mathsf{AX}(1;0,0);\mathsf{S}^{10};\mathsf{MP}(0).$$

The first MP fails, since $\mathsf{I}(t,1)>0$ while cache is zero; it produces $(0,w,1,0)$. AX has $A=c=1$, gate zero, and saved top zero. Ten transfers give $(\mathsf{Q}_1 0,w,0,0)$, after which MP fails because $\mathsf{I}(\mathsf{Q}_1 0,0)>0$. Thus DISCARD is B_0, returning $(0,w,0,0)$ without halting.

When top, counter, and cache are zero, let $J(0)$ be the empty word and $J(u) = \mathsf{MP}(u)$ for $u > 0$. It sets the counter to u without changing the stack. Omitting MP at $u = 0$ is essential: the alias $\mathsf{I}(0,0) = 0$ would make it succeed and push a zero.

7.2 Axiom and generalization blocks

For a retained row $R_j(a,b,c)$, $1 \le j \le 7$, use

$$\text{DISCARD}; \mathsf{AX}(c;a,b); \mathsf{S}^{j+1}; H^{9-j}.$$

The shifts put row j on top. Each H preserves that non-1 top and removes one generated stack entry; the last removed entry is saved zero. The original stack is restored with zero counter and cache.

For the dominated row $L(a,b,c,u)$ with $u \le a$, use

$$\text{DISCARD}; J(u); \mathsf{AX}(c;a-u,b); \mathsf{S}^9; H.$$

For an enabled full bundle, use

$$\text{DISCARD}; \mathsf{AX}(c;a,b); H^{10}.$$

The latter retains every part of the original gated bundle.

Given B_X, theorem generalization is compiled by

$$B_{\mathsf{Q}_i X} = B_X; \text{PUSH}; J(i); \mathsf{S}; \mathsf{AX}(i;0,0); \mathsf{S}^{10}.$$

After PUSH and $J(i)$, the single transfer maps $(0,[X;w],i,0)$ to $(0,w,i,X)$ without testing the newly loaded X. AX has $U = A = c = i$, zero gate, ninth head $\mathsf{Q}_i X$, and saved zero. Ten transfers return exactly $(\mathsf{Q}_i X, w, 0, 0)$. This includes $i = 0$ and $X = 1$.

7.3 Major-premise-first modus ponens

For $M = \mathsf{I}(A,B)$, define

$$\begin{aligned} B_B = B_M; \text{PUSH}; B_A; \text{PUSH};& \\ \mathsf{S}; \mathsf{S}; \mathsf{MP}(B); \mathsf{AX}(B;0,0); \mathsf{S}^{11}.& \end{aligned}$$

Unless a permitted earlier halt occurs, the first four blocks produce $(0,[A,M;w],0,0)$. Two transfers produce $(A,w,0,M)$; if the second one instead sees $A = 1$, it has already succeeded by halting. MP now succeeds by exact numerical equality and returns $(B,[0;w],B,0)$.

The following AX has gate zero and saves B. Ten transfers leave $(\mathsf{Q}_B 0,[0;w],0,B)$. The eleventh halts if $B = 1$, and otherwise returns $(B,w,0,0)$. When $A = B = 0$, the major is literally zero and MP deliberately succeeds; the final transfer removes precisely the zero it pushed. No hidden nonzero-code assumption enters the calculation.

Induct on a finite native proof, expanding a finite proof DAG to a proof tree if necessary. Compiling the major first creates no dependency cycle. The preceding cases prove (Strong). In particular, $B_1; H$ halts from every $(t,w,0,0)$.

7.4 Raw wire encoding and cofinal successful candidates

Let $F(fs,r)$ right-fold a field list by Cantor pairing over r, and put

$$\text{tail}([]) = 0, \qquad \text{tail}(k :: L) = 1 + F(f(k), \text{tail}(L)).$$

A positive residual is decremented before its next command; the first command of a fresh candidate is read directly from the candidate number. The successor outside F is therefore required for a continuation, but absent at fresh fetch.

Let $R = \mathrm{tail}(B_1; H)$. For every natural Y, the candidate

$$N_Y = \mathsf{P}(2, \mathsf{P}(Y, \mathsf{P}(1, R))) \tag{5}$$

starts with $\mathsf{AX}(0; 1, Y)$, which resets counter and cache from any retained store. The strong compiler accepts any resulting top and stack, so the fixed suffix halts. Since $N_Y \geq Y$, successful candidates are unbounded.

Each nonhalting positive residual strictly decreases: its next value is a right projection after one, two, or three unpairings of its predecessor. All arithmetic commands terminate on natural inputs. Every nonhalting candidate therefore drains, and ordinary candidate increment by one reaches any chosen N_Y above the initial cutoff. Completeness needs neither a well-formedness filter nor a fairness hypothesis.

8 Protected credit for the actual scheduler

Here is the all-natural argument needed for the opaque startup tail. Let f denote the right Cantor projection and $T_s = s(s+1)/2$. Let $W(n)$ be the entire syntactic command word of fresh candidate n, ignoring possible semantic halts. Set $R(0) = []$ and $R(r) = W(r-1)$ for $r > 0$. If $\mathrm{unpair}(n) = (a, y)$, then

$$W(n) = \begin{cases} \mathsf{S}; R(y), & a = 0, \\ \mathsf{MP}; R(f(y)), & a = 1, \\ \mathsf{AX}; R(f(f(y))), & a \geq 2. \end{cases} \tag{6}$$

The command parameters are irrelevant to credit, but their arity and continuation decrement are not.

Assign weights $\mathsf{S} = -1$, $\mathsf{MP} = 0$, $\mathsf{AX} = 10$. Successful MP's additional truth push is conservatively ignored. Write $b(n)$ for total weight, $B(k) = \sum_{n<k} b(n)$, and $g(0) = 0$, $g(r) = b(r-1)$ for $r > 0$. By induction $W(n)$ has at most $n+1$ commands, so every prefix of $R(r)$ loses at most r units.

Lemma 2 (Cantor prefix pullback)**.** *If a sequence has nonnegative initial partial sums, its composition with f, and then with f^2, has nonnegative initial partial sums.*

Proof. In increasing argument order, f concatenates the lists $[0], [0,1], [0,1,2], \ldots$. Every finite prefix consists of full lists followed by an initial partial list. Each corresponding sum is nonnegative. Repeat for f^2. □

Lemma 3 (Later diagonals require no new credit)**.** *Every Cantor diagonal $s \geq 2$, read in increasing candidate order, has nonnegative weight at every command prefix, relative to its start. Moreover $B(k) + k \geq 0$ for every k, and $B(T_s) \geq -3$.*

Proof. The exceptional words are $W(0) = \mathsf{S}$, $W(1) = \mathsf{MP}$, and $W(2) = \mathsf{SS}$. The first values of B are therefore $0, -1, -1, -3$, supplying the induction base.

Use the shifted sequence $q(r) = g(r) + 1$. For $k > 0$,

$$\sum_{r<k} q(r) = k + B(k-1) \geq 0.$$

On diagonal $s \geq 2$, the first $s-1$ candidates begin with AX, the next begins with MP, and the last with S. The first k AX candidates have total weight at least $9k$ by the twice-pulled-back inequality.

Before AX at position y, the balance is at least $9y$; after its initial $+10$, any suffix prefix loses at most $f^2(y) \le y$. Thus all AX prefixes are safe.

After that portion, balance is at least $9(s-1)$. The MP suffix loses at most $s-1$, and the last S and its suffix at most $1+s$. Hence every prefix through those last two words is bounded below by

$$9(s-1) - (s-1) - 1 - s = 7s - 9 > 0.$$

All pullback indices used here belong to earlier diagonals, so the induction is well founded. The new diagonal is prefix-safe; combining $B(T_s) \ge -3$ with $k \ge 3$ gives $B(k) + k \ge 0$ at its new boundaries. This closes the joint induction. □

Starting at candidate 1, the exceptional prefix is $W(1)W(2) = \mathsf{MPSS}$, with minimum weight -2. No later diagonal makes this worse. Thus the exact *conservative* debt at the actual cutoff is

$$d(1) = 2.$$

This is a statement about the syntactic weight bound; it does not claim every actual execution attains that debt, since successful MP may add heads.

Maintain an explicit finite list of theorem heads above a fixed opaque tail J. AX prepends ten heads, MP prepends a theorem zero on success and none on failure, and S consumes one head. With two initial heads the list has enough credit before every S. Top and cache remain theorem-valued; the S guard can therefore halt only on a native theorem 1. An explicit list is needed because $\mathsf{P}(0,0) = 0$ does not uniquely encode list length. The invariant uses only $\mathrm{unpair}(\mathsf{P}(x,v)) = (x,v)$.

ST215 begins the ordinary schedule with ten heads, so at least eight explicit heads remain over $J = 2$ at every boundary. Its unsafe numerical tail never becomes visible. The full inherited proof is included as `proofs/protected_credit.md`.

9 The consistency equivalence

Proof of Theorem 1. The exact finite blank trace reaches the represented interior AX frame of Section 6. The matching source prefix and all-natural AX contract complete it to ten theorem heads above tail 2, with next candidate 1 and empty residual. The universal physical simulation transfers this conclusion to the literal table.

For soundness, the credit theorem protects the tail, and the command equations preserve theorem-valued top, cache, and explicit heads. The only halt tests cache equal to 1. Thus binary halting gives $\mathcal{D} \vdash 1$, and (1) gives a finite ZFC refutation.

Conversely, a ZFC refutation gives $\mathcal{D} \vdash 1$ by the same inherited calibration. The strong compiler produces $B_1; H$. Equation (5) supplies arbitrarily large successful candidate numbers, irrespective of the retained stack. The ordinary exhaustive scheduler reaches one unless it halts earlier. Source termination of that successful candidate lifts to finite binary halting.

These are finite calculations, primitive-recursive proof and word transformations, and induction on natural numbers, finite lists, and derivations. They are formalizable in ZFC without assuming its consistency. Negating the resulting halting equivalence yields the displayed nonhalting equivalence. □

In particular, this result is not an unconditional proof that ST215 never halts. Assuming the usual consistency of ZFC, it does not halt and ZFC cannot prove that fact. No claim about ZFC's inability to prove a false halting statement from mere consistency is needed here.

10 Other formalisms and research results

The search was pursued at two levels: changing the operational calculus and changing its physical realization. The positive machine uses the inherited calibrated numerical foundation, but dispenses with a separate DROP command. The following results explain why some more radical small interfaces fail and identify a different abstract scheduler.

10.1 Rotating pop has a structural obstruction

One incoming experiment replaces transfer by

$$\mathsf{R}(t, [x; v], d, h) = (x, v, d, t),$$

halting instead when $h = 1$ or $x = 1$. A compact runtime does not make this interface refutationally complete.

Call an adjacent pair (m, a) productive when $m = \mathsf{I}(a, b)$ for some $b > 0$. Consider the spine consisting of cache, top, then successive stack heads. A spine is blocked if it contains neither 1 nor a productive pair. The all-zero spine is blocked. R drops its first spine entry. An MP success on a blocked spine must have consequent zero. The literal AX row order creates no productive adjacency: the ten necessary mismatches follow from injectivity and incompatible outer tags modulo four. Thus every command word from a zero store remains blocked and never halts.

This excludes the proposed strong compiler even for the fixed native theorem

$$20 = \mathsf{I}(0, \mathsf{Q}_0 0) = \mathsf{I}(0, 2).$$

Truth, GEN, the recovered K schema, and MP prove it. A returned store $(20, 0, 0, 0)$ would contain the productive adjacent pair $(20, 0)$, while the alternative successful outcome, halting, is also excluded. This obstruction is unconditional and does not assume Con(ZFC). Appendix B supplies the full all-natural proof and its precise scope for the archived physical experiment.

10.2 A scratch increment changed the return target

The incoming scratch-biased short-transfer experiment inserted CI6 before CC3. Its earlier return address 10011 became 100111. The extra right ancestor makes CC3 return to selector ordinal 23 rather than scheduler ordinal 0. This is a semantic change, despite an unchanged opcode.

After AX, selector and cache are zero. CI6 leaves scratch $X = 1$; the misdirected return immediately enters S. The copy/restore loop injects that scratch unit into cache, and NIP halts. The all-natural suffix length is $2t + 15$ source transitions; for the actual initial zero gate it is 15. The exact source checks and proof are in Appendix C. The physical primitive checker can faithfully implement this faulty source and still pass every runtime case. It cannot prove the advertised higher-level parser.

10.3 Consecutive successful candidates tolerate skipped numbers

A different *abstract* parser can be calibrated if its equations are implemented correctly. Let a bit b become zero after S and one after MP or AX. Read the next command from $L - 1 + b$ when the continuation pointer L is positive. Suppose fresh candidate numbers increase by one or two. Unbounded successful candidates alone are insufficient: a sparse search could skip all of them.

For any $r > 0$, define

$$a = \mathsf{P}(2, \mathsf{P}(1, \mathsf{P}(0, r))), \quad z = \mathsf{P}(1, a), \quad n = \mathsf{P}(1, z).$$

Candidate n runs $\mathsf{MP}(1);\mathsf{AX}(0;0,1)$ and reaches $(L,b)=(r,1)$. The identity $\mathsf{P}(1,z)+1=\mathsf{P}(0,z+1)$ makes candidate $n+1$ run $\mathsf{S};\mathsf{MP}(2);\mathsf{MP}(0)$ and reach the same parser state, or halt sooner. The arithmetic stores need not agree; the common suffix begins with a reset.

Encode a successful reset-plus-refutation suffix with an arbitrary positive unused tail q. This gives cofinally many *consecutive* successful pairs. Any increasing schedule with gaps at most two meets each sufficiently late pair. Positive continuation pointers strictly decrease, so the search actually reaches its subsequent selected candidates.

The positive tail is essential: with zero tail, the biased encoding of a final S can itself be zero, and the parser would skip it. The complete arithmetic proof is Appendix D.

There is also a separate protected-credit theorem for the hypothetical parser in `research/biased_credit_proof.md`. Writing $D(n)$ for a word's maximum debt, it proves $2^{D(n)}\le n+2$ and $\sum_{n<k}2^{D(n)}\le 4k$ by Cantor-diagonal induction. The pullback bound then makes every selected diagonal $s\ge 5$ prefix-safe for *any* selection with gaps one or two. The explicit initial schedule from candidate 1, bit 1, also has nonnegative balance. With ten theorem heads this gives protected-tail soundness. Combined with the consecutive-pair compiler, these results calibrate the intended abstract search equations.

No physical implementation of those biased equations is claimed here. The archived scratch-biased machine does not implement them, and ST215 does not use them. Their potential value is that an implementation may retain a scratch bit instead of paying to erase it, while preserving the only required property: equiconsistency. A source and table implementing the equations remain a separate research problem.

11 Physical comparison and the remaining 214-state problem

The comparison corpus contains 78 distinct 215-state *byte files*, under 332 original file names, drawn from the pinned K215 directory and twelve incoming archives. Those are not claimed to be 78 mutually nonisomorphic machines. Every input is included verbatim with its origin and SHA-256.

A rooted deterministic labelled-graph traversal compares write bits, directions, read-symbol edges, and halt targets. It finds no state-relabeling isomorphism between ST215 and any corpus table, including global left–right reflection. All 215 working states are reachable in this abstract graph. Graph reachability does not assert blank-run reachability.

As a control, restore just the append-one write at 198/0. The resulting precursor still has no direct or reflected match. Thus the difference is not solely the refill bit. Its action differs from published K215 after the abstract read word

$$000000100100000000001001:$$

the control writes 1 and moves right where K215 writes 0 and moves left. This read word is a graph witness, not a claimed physically realized tape history.

The canonical rooted graph hash of the final native table is

`28eceb93cebfd2529c9140f0bf6c6d7c3bf110f921cf584958caa61efa5e02f4`

The corpus, algorithm, receipts, and independent review are in `research/comparison/`. This establishes novelty relative to the inspected corpus, not global historical priority.

Bounded attempts to save the final state found no accepted 214-state machine:

Search family	Specifications	Survivors
K215 local folds, radius 5, fuel 97	316,292	0
K215 local folds, radius 7, fuel 97	316,292	0
Short-layout local folds, radius 7, fuel 97	309,444	0
Concrete primitive row-supplier quotients	91,164	0

The short-layout searches use the append-one precursor, not the final append-zero table. The quotient screen first passes all 6,151 baseline concrete primitive cases. Its finite arithmetic fixtures, tape windows, and budgets do not prove universal validity for any survivor, and failure to find a survivor proves no lower bound. Exact inputs, portable drivers, original receipts, and limits are in `research/physical/`.

The usable gain is a physically valid short transfer source. Future state reduction can begin with its 35-leaf body and calibrated startup. The successful repair suggests searching over source contexts jointly with state allocation: a primitive's reusable physical implementation can depend on ancestor bits that a source DAG count alone does not record.

12 Reproduction, artifacts, and inherited boundaries

The packet is designed to be copied into the research directory without requiring a repository checkout to run its main checks. Python 3 and its standard library suffice. Assertions must be enabled.

```
python3 -I verify.py --work /tmp/st215-finite
python3 -I verify.py --runtime --work /tmp/st215-full
```

The selected output directory must not already exist. The first command verifies the manifest, deterministic reconstruction, both CFG resolvers, source identities, exact native/dense startup, the source AX prefix, the fixed propositional certificate, saved runtime metadata, and the independent graph comparison. The second also rederives every symbolic primitive case. On the preparation machine the latter symbolic derivation takes a few minutes; timing is not a theorem.

Path	Purpose
`machine/`	Exact native/dense ST215 tables and explicit bijection.
`source/`	The complete ordered primitive occurrence list.
`checks/`	Reconstruction, source, startup, logical and symbolic checkers.
`evidence/`	Original and portable receipts, full startup deltas and source prefix.
`proofs/`	Strong compiler, credit proof, and independent source review.
`appendices/`	Include-ready TeX for independent review and additional results.
`research/`	Portable comparison corpus, rejected-source audit, abstract parser lemmas, bounded searches.
`reference/transfer-parent/`	Unchanged inherited parent packet, including its proof texts, licenses, and exact inputs.
`provenance/`	Pinned upstream identities, inherited foundations, and attribution.

The 3,728-node checker proves the stated finite universal propositional recovery. The source checks prove finite syntax and control identities. The startup checker proves a finite literal run. The symbolic checker proves arbitrary-natural primitive contracts by sound induction rules. The written compiler, credit, and inherited calibration connect these levels. None of those individual checks should be advertised as proving all the others.

The new runtime wrapper uses the inherited proof engine, with a compact retained interface and new exact ST215 inputs. Its mathematical rules are unchanged. This packet intentionally executes

the included verifier code; attribution to its earlier authors is retained. Historical prose inside the unchanged parent packet describes that parent's execution policy and is not a claim of newly authored code here.

The source snapshot is commit

`75716498437bfc7291d5900cab2983a11fd54bac`

of ProveIt. The exact K215 ordinary proof, dominated-calculus proof, transfer compiler, credit proof, and original source licenses are identified in the provenance. The new machine theorem reuses their stated arithmetic and numerical-calculus results. No foundation is inferred merely from successful execution of a finite script.

Build the PDF with two runs of `pdflatex -halt-on-error report.tex`. The transition appendix is generated from the dense table by the included deterministic script. The table files themselves are the primary machine specification.

References

[1] Vladimir Reshetnikov, *ProveIt: Smaller machines for Con(ZFC)*, pinned repository snapshot, 21 October 2026. https://github.com/VladimirReshetnikov/ProveIt/blob/75716498437bfc7291d5900cab2983a11fd54bac/Computability/BusyBeaver/Research/ZFC/README.md

[2] *Ordinary proof of the combined 215 machine*, same snapshot. https://github.com/VladimirReshetnikov/ProveIt/blob/75716498437bfc7291d5900cab2983a11fd54bac/Computability/BusyBeaver/Research/ZFC/candidate-k215/proof.md

[3] *Dominated-rho8 alternative: proof*, same snapshot. https://github.com/VladimirReshetnikov/ProveIt/blob/75716498437bfc7291d5900cab2983a11fd54bac/Computability/BusyBeaver/Research/ZFC/candidate-k218/alternatives/dominated/proof.md

[4] *T215 Transfer*, incoming research packet, `docs/incoming/T215_Transfer_2026-10-20.zip`, same snapshot. Its frozen transfer parent is included verbatim under `reference/transfer-parent/`; particularly `proofs/transfer_compiler.md`, `proofs/protected_credit.md`, and its source-calibration provenance.

[5] *MP0 Transfer 215*, incoming research packet, `docs/incoming/MP0_Transfer_215.zip`, same snapshot. The rotating-pop experiment is the target of Appendix B.

A Independent ST215 bootstrap and calibration review

The append-zero, start-zero ST215 table passes an independent literal blank-tape replay and a complete synthetic-AX prefix comparison. The result closes the dirty-tail initialization obligation for the ordinary short-transfer calculus. The independent check does not infer the initialization from a plausible local pattern: it compares the entire finite set of one-cells and all nine logical registers.

A.1 Exact finite evidence

The literal table has 430 defined transitions, one for each symbol in each of 215 nonhalting states. State 254 is its only undefined target. Starting in state zero with a blank tape and head at zero, after exactly 70,506 binary transitions it reaches the following complete frame:

Quantity	Value
Native dispatcher state	84
Head position	-2
Header position and gap	62, gap zero
Source ordinal	58, primitive $\mathsf{CI}(7)$
Source path	`00110010010110010`
Logical register order	$(N, T, W, A, D, H, X, Y, L)$
Logical register values	$(1, 0, 0, 0, 0, 0, 0, 18, 0)$
Spare blocks	63, all zero-valued
One-cells on the entire tape	100, from -3 through 225

The phrase "from -3 through 225" gives the minimum and maximum, not a contiguous run. The full 100-cell set appears in the JSON receipt.

Here is an independent description of the reconstructed frame. Relative to the dispatcher head, place a guard at -1, the one-bits of the source path at their zero-based positions, and a header at 64. The logical-to- physical block permutation is

$$(6, 2, 0, 4, 3, 7, 8, 5, 1).$$

Beginning at position 65, each block is a separator zero followed by $v + 1$ one-cells for its value v. Encode the nine permuted logical registers followed by the 63 zero spare blocks, and translate by -2. This construction gives exactly the tape obtained by direct execution; there are no unexamined one-cells outside the frame.

Two independently formulated source-control resolvers, one using subtree interval endpoints and one using arithmetic cuts of binary paths, agree on all 4,732 primitive leaves. The resolved graph is also identical to the complete unrotated context-root short-transfer graph. In particular, the AX/MP continuation at ordinal 4696, $\mathsf{CC}(3)$, returns to ordinal zero. This is the continuation that the archived scratch-biased experiment failed to preserve. Independent subtree interning counts 201 distinct source DAG nodes.

The source trace starts at AX entry ordinal 25 with

$$(1, 0, 2, 0, 0, 0, 0, 0, 0).$$

After exactly 177 source transitions it reaches ordinal 58 with

$$(1, 0, 0, 0, 0, 0, 0, 18, 0),$$

exactly the register store of the binary frame. The complete 178-row trace, including both endpoints, is retained in the receipt. This synthetic prefix is a continuation witness: the argument does not

claim that the actual blank run previously passed through that AX-entry store. Determinism identifies their futures once they reach the same ordinal and complete logical store, and the whole-tape simulation identifies that source future with the actual native future.

The independent table hash agrees with the complete runtime receipt. That receipt reports 6,151 guarded cases, all 430 literal rows covered, 1,617,630 symbolic rules, no failures, and minimum duration five. The independent finite checker here does not rerun those symbolic cases; its role is to establish the exact initial frame and source-prefix bridge to which that all-natural simulation theorem applies.

A.2 Completion of the partial AX

Write P for Cantor pairing, $[x; w] = P(x, w)$, $I(x, y) = 4P(x, y)$, and $Q_i x = 4P(i, x) + 2$. Truth has code zero and falsity has code one. For a command store use (t, w, d, h) for top, encoded stack, additive counter and cached formula.

The synthetic AX has parameters

$$(t, w, d, h) = (0, 2, 0, 0), \qquad (c, x, y) = (0, 0, 0).$$

Its inherited all-natural arithmetic contract terminates and yields

$$(t, d, h) = (0, 0, 0), \qquad N = 1, \quad L = 0,$$

with stack

$$[r_1(0, 0, 0), \ldots, r_7(0, 0, 0), L(0, 0, 0, 0), Q_0 0, 0; 2].$$

The first eight displayed heads are literal axiom instances, the ninth is the generalization $Q_0 0 = 2$ of truth, and the tenth is saved truth. Thus all ten explicit heads are theorems of the calibrated numerical calculus. The gate is zero because $c = A = B = 0$. The remaining tail is raw integer two; it is not assumed safe. Indeed $2 = P(0, 1)$ and $1 = P(1, 0)$, so that tail contains a falsity head.

Only the finite 177-step prefix is executed in the bridge check. The termination and result of the much larger remaining AX are supplied by its arithmetic contract. Literal simulation of the enormous encoded output is unnecessary and is not claimed.

A.3 Protected-tail soundness

The actual source implements the following short transfer:

$$\mathsf{S}(t, [x; v], d, h) = \begin{cases} \text{halt}, & h = 1, \\ (h, v, d, x), & h \neq 1. \end{cases}$$

It tests only the incoming cache. In particular, it is distinct from the longer composite transfer that also tests the incoming top. MP and AX retain their inherited equations. The field arities are one for S (tag zero), two for MP (tag one), and three for AX (tags at least two).

The all-natural scheduler credit proof applies to exactly those tags and arities. Give S weight -1, MP weight zero, and AX weight ten, ignoring the additional zero head of a successful MP. Starting with fresh candidate one and an empty proof pointer, every finite schedule prefix has weight at least -2. The exceptional initial schedule is $W(1); W(2) = [\mathsf{MP}, \mathsf{S}, \mathsf{S}]$; every later complete Cantor diagonal is prefix-safe. Thus the ten theorem heads produced above leave at least eight protected explicit theorem heads forever.

For clarity, the credit proof uses the exact recurrences

$$\begin{aligned} W(n) &= \mathsf{S}; R(y) && \text{if } \operatorname{unpair}(n) = (0, y), \\ W(n) &= \mathsf{MP}; R(f(y)) && \text{if } \operatorname{unpair}(n) = (1, y), \\ W(n) &= \mathsf{AX}; R(f(f(y))) && \text{if } \operatorname{unpair}(n) = (a, y),\ a \geq 2, \end{aligned}$$

where f is right projection, $R(0)$ is empty, and $R(p) = W(p-1)$ for $p > 0$. In particular, AX takes two further projections after its selector, not three. Its proof is an induction over complete Cantor diagonals, not a bound inferred from a finite simulation.

Maintain a finite explicit list of theorem heads over the fixed tail two. AX adds ten such heads. MP preserves the list or adds theorem zero. Each S has at least one explicit head available before popping, because the conservative count after its decrement remains at least eight. Thus it cannot expose the tail. Its new top and cache are theorems; a halt witnesses theorem one. This argument counts explicit wrappers, not numerical code size, so it includes the alias $P(0,0) = 0$. No assertion that all raw integers are formulas, or that the buried tail consists of formulas, is used.

A.4 Refutational completeness

The inherited strong compiler is already proved for this exact short S, rather than by treating it as equivalent to old NIP. Put $D = \mathsf{AX}(0;0,0)$ and

$$H = D;\mathsf{S}^{11}.$$

It satisfies, from every natural store,

$$H(t,[x;w],d,h) = \begin{cases} \text{halt}, & t = 1, \\ (t,w,0,x), & t \neq 1. \end{cases}$$

The first ten S tests inspect the new non-falsity rows; the eleventh tests the saved incoming top. For every theorem X, the compiler constructs a finite block B_X which, from every $(t,w,0,0)$, either halts or returns exactly $(X,w,0,0)$. The arbitrary-store quantifier is essential here: it covers any preserved stack tail and every top produced by a resetting AX.

If the calculus proves one, the fixed word $B_1;H$ therefore halts from every such reset store. Let $F(\vec{a},r)$ denote right-nested pairing of a finite field list over residual r, and define

$$\text{tail}([]) = 0, \qquad \text{tail}(k :: V) = 1 + F(\text{fields}(k), \text{tail}(V)).$$

The positive residual is decremented before decoding its command, so this is the exact continuation encoding. Put $R = \text{tail}(B_1;H)$. For every $Y \in \mathbb{N}$, the fresh candidate

$$N_Y = P(2, P(Y, P(1,R)))$$

begins with $\mathsf{AX}(0;1,Y)$ and then executes $B_1;H$. The initial AX resets the counter and cache, and the strong compiler handles its arbitrary resulting top and stack. Therefore every N_Y is successful from every retained store, and $N_Y \geq Y$ gives cofinality.

Every nonhalting residual command decreases the positive proof pointer: the next residual is a right projection of its predecessor, with further projections for MP and AX. Each candidate therefore drains in finitely many commands, and the unchanged next-candidate counter advances by one. From next candidate one the scheduler must reach a successful N_Y, unless it halts sooner. This proves refutational completeness without any bound on the size of a proof or its code.

A.5 Result and exact proof boundary

Combining these arguments gives

$$\mathsf{ST215} \text{ halts on blank tape} \iff \text{the inherited dominated numerical calculus proves 1.}$$

The established calibration of that unchanged calculus to ZFC then gives, by ordinary reasoning formalizable in ZFC,

$$\text{ZFC} \vdash (\mathsf{ST215} \text{ does not halt on blank tape} \longleftrightarrow \text{Con(ZFC)}).$$

This conclusion uses the inherited arithmetic contracts and calculus calibration, the all-natural protected-credit and strong-compiler proofs, the exact finite startup/AX bridge, and the complete symbolic primitive simulation. Each supplies a different boundary; no finite test replaces an unbounded theorem. The source rotation preserves the full control graph, and the bootstrap passes into an unchanged AX, so neither step requires a new logical axiom schema or a claim that junk codes decode to ordinary formulas. This review does not claim that a new Lean or Rocq endpoint has been compiled.

The accompanying independent files are

`check_st215_bootstrap.py`
`st215_bootstrap_independent.json`

They bind the source and native bytes by SHA-256 and include the complete finite witnesses.

B A productive-adjacency obstruction for rotating pop

The archived packet `MP0_Transfer_215.zip` contains a physically validated experimental 215-state table, `research/rotate-pop215/R.native.tm`, whose refutational completeness is explicitly left open. Its command

$$\mathsf{R}(t,[x;v],d,h)=(x,v,d,t)$$

halts instead when $h=1$ or $x=1$. Here $[x;v]=P(x,v)$, with Cantor pairing P, and $\mathrm{unpair}(0)=(0,0)$. The following argument shows that the missing compiler cannot be repaired while keeping this interface and the literal AX row order. It uses no consistency assumption, no model of set theory, and no theoremhood invariant.

Put $I(a,b)=4P(a,b)$, $Q_ab=4P(a,b)+2$, $E(a,b)=4P(a,b)+3$, $M(a,b)=4P(a,b)+1$, $F=1$, and $NX=I(X,F)$. Define $\exists_aX=NQ_aNX$. The actual AX output is

$$(G,[r_1,r_2,r_3,r_4,r_5,r_6,r_7,L,Q_Uh,t;w],0,0),$$

where $U=d$, $A=d+x$, $B=h+y$, $C=c$, the r_j are exactly

$$\begin{aligned}
r_1&=\exists_AE(A,B),\\
r_2&=I(Q_AI(B,C),I(Q_AB,Q_AC)),\\
r_3&=I(E(A,B),I(M(C,A),M(C,B))),\\
r_4&=I(Q_AQ_BC,Q_BQ_AC),\\
r_5&=I(E(C,A),I(M(C,B),M(A,B))),\\
r_6&=I(\exists_AQ_AC,C),\\
r_7&=I(E(C,A),I(E(C,B),Q_{M(A,B)}E(A,B))),\\
L&=I(I(I(C,U),A),I(I(A,C),I(B,C))).
\end{aligned}$$

The gate G is either zero or the literal full bundle

$$G=I(I(V,J),F),\qquad V=I(M(A,B),Q_CM(A,B)).$$

The contents of J are immaterial to the argument: the full inherited term is retained. Thus the result does not weaken the underlying theory.

Call a pair (m,a) *productive* when

$$\exists b>0\quad m=I(a,b).$$

For a command store, consider the ordered spine $h,t,x_1,x_2,\ldots$, where x_i are successive heads of its encoded stack. A spine is *blocked* when none of its entries is 1 and none of its adjacent pairs is productive. The all-zero spine is blocked. This definition is numerical; it does not require that an entry decode to a conventional formula.

The AX adjacency lemma. Every newly introduced adjacent pair of the AX spine

$$0, G, r_1, r_2, r_3, r_4, r_5, r_6, r_7, L, Q_U h, t, x_1, \dots$$

is unproductive. The existing suffix beginning with t is unchanged. For the pair beginning with zero this follows from the exact implication alias $I(a, b) = 0 \iff a = b = 0$. For the remaining pairs, injectivity of I reduces a possible match $m = I(a, b)$ to equality between the antecedent of m and a. The following table gives the contradictory outer tags; nested comparisons use injectivity once more.

Pair	Required equality	Contradiction modulo 4
(G, r_1), enabled gate	$V = Q_A N E(A, B)$	$0 \neq 2$
(r_1, r_2)	$Q_A N E(A, B) = r_2$	$2 \neq 0$
(r_2, r_3)	$Q_A I(B, C) = r_3$	$2 \neq 0$
(r_3, r_4)	$E(A, B) = r_4$	$3 \neq 0$
(r_4, r_5)	$Q_A Q_B C = r_5$	$2 \neq 0$
(r_5, r_6)	$E(C, A) = r_6$	$3 \neq 0$
(r_6, r_7)	$Q_A N Q_A C = E(C, A)$	$2 \neq 3$
(r_7, L)	$E(C, A) = L$	$3 \neq 0$
$(L, Q_U h)$	$I(I(C, U), A) = Q_U h$	$0 \neq 2$
$(Q_U h, t)$	$Q_U h = I(t, b)$	$2 \neq 0$

For a disabled gate $G = 0$, the zero-pair argument applies. All generated entries are different from 1: the gate and eight rows have tag zero, and $Q_U h$ has tag two. This includes the possible alias $L = 0$; no positivity of L is assumed. The inequality $U \leq A$ is not needed.

Preservation theorem. Every finite word of the rotating-pop commands, MP and AX preserves a blocked spine and never halts when begun at a blocked spine. In particular, this holds for every finite word from $t = h = w = 0$, with arbitrary additive counter and arbitrary raw command parameters.

Proof. AX preserves the property by the adjacency lemma. Rotating pop maps the spine $h, t, x_1, x_2, \dots$ to $t, x_1, x_2, \dots$, a suffix. Its two tested entries are not 1. For MP put $y = d + x$. If it succeeds, $h = I(t, y)$, so blockedness forces $y = 0$. The returned spine is $0, 0, 0, x_1, x_2, \dots$. If it fails, the returned spine is $0, 0, x_1, x_2, \dots$. Every new pair beginning with zero is unproductive, and every new entry is zero. In either case blockedness is preserved. Induction on the word proves the claim. This proof also covers every numerical zero-stack alias. □

The actual dirty-tail initialization. The preceding argument has a finite-prefix form. For any fixed raw tail J, maintain a finite explicit head list X over J such that the finite spine $[h, t] + X$ is blocked. AX adds ten explicit heads, R consumes one, and MP adds either zero heads or one zero head. Whenever the current explicit head list is nonempty, the same preservation proof applies. Thus an external scheduler credit bound protects *blocked* heads just as it protects theorem-valued heads; the latter interpretation is unnecessary here.

For the archived R table, the supplied exact blank startup reaches the interior of the unchanged AX at source ordinal 55, with registers $[1, 11, 2, 0, 0, 0, 0, 0, 0]$. Its recorded AX-prefix match completes to next candidate 1, empty proof pointer, $t = d = h = 0$, and

$$[r_1(0, 0, 0), \dots, r_7(0, 0, 0), L(0, 0, 0, 0), Q_0 0, 0; 4].$$

The displayed ten-head prefix is blocked by the lemma, independently of the tail $4 = P(1, 1)$. The unchanged wire arities are one field for R (tag zero), two fields for MP (tag one), and three fields for AX (tags at least two). The inherited all-natural Cantor-diagonal credit proof assigns

weights $-1, 0, 10$ and proves that every schedule prefix from candidate 1 has weight at least -2. Consequently at least eight explicit blocked heads remain throughout the actual execution. The buried 1 heads are never reached. The source therefore never halts, by an ordinary proof formalizable in ZFC. Combining the supplied positive-duration primitive simulation and the complete blank prefix gives the same unconditional nonhalting statement for the exact R table.

This last exact-table conclusion is conditional on those independently specified physical certificates; the new proof here establishes the command-level obstruction and its protected-prefix transport. It does not rerun the large physical certificates. The checker accompanying this section verifies all ten literal adjacency obstructions and the exact numerical gate/row formulas used in this proof. It does not claim to be a proof-assistant kernel check.

A fixed theorem defeats the proposed strong compiler. The calibrated numerical calculus proves

$$X = I(0, Q_0 0) = I(0, 2) = 20.$$

Indeed truth gives zero, GEN gives $Q_0 0 = 2$, and the derived propositional K schema gives $2 \to (0 \to 2)$; one MP gives X. The dominated calculus inherits that K schema from its established propositional recovery certificate. A strong block for X, applied to the all-zero store, would have to halt or return $(20, 0, 0, 0)$. The preservation theorem excludes halting. The alleged return has spine $0, 20, 0, 0, \ldots$, containing the productive adjacent pair $(20, 0)$ because $20 = I(0, 2)$ and $2 > 0$. Hence the return is also impossible. No strong compiler for all numerical theorems exists, even for this fixed elementary theorem; this conclusion does not use any consistency or incompleteness assumption.

Consequence for refutational completeness. More generally, no block can take a blocked all-zero store to a return state with top 1 or to a halt. If a refutation-complete compiler for the calibrated ZFC calculus existed and were provable in ZFC, the unconditional nonhalting proof would yield Con(ZFC) in ZFC. Under the usual consistency assumption this contradicts the second incompleteness theorem. The obstruction is specifically caused by preserving the literal AX order while replacing transfer-pop with rotating pop. Changing that order, emitting an additional bridge pair, or changing the return map would need a new analysis.

C A control-flow error in the archived scratch-biased experiment

The scratch-biased 215-state experiment in `T215_Transfer_2026-10-20.zip` has a successful quantified primitive-runtime certificate, but its README's high-level command description is incorrect. Replacing a continuation leaf by a two-leaf subtree changed the continuation's target. This defect is visible in the literal source independently of physical allocation.

The pre-insertion continuation was $\mathsf{CC}(3)$ at binary path 10011. Its three right ancestors are at indices $0, 3, 4$, so the third outward right ancestor is the root and the continuation returns to source ordinal zero. The replacement subtree $(\mathsf{CI}(6), \mathsf{CC}(3))$ places the continuation at 100111. There are now four right ancestors, at indices $0, 3, 4, 5$. Its third outward right ancestor has prefix 100, whose first leaf is source ordinal 23. Thus the literal edge is

$$4697 : \ \mathsf{CC}(3) \longrightarrow 23,$$

not an edge to zero. The added right branch must be counted in a relative continuation instruction.

Exact unconditional halting suffix. At ordinal 4696, assume only that logical registers $A = H = X = 0$; write the arbitrary current top as $T = t$. Other logical registers are irrelevant to the following suffix. The actual execution is:

1. Ordinal 4696 increments scratch X to one; ordinal 4697 returns to 23.
2. The zero selector A takes the zero branch at 23, reaching ordinal 4698, the top-clear loop. That loop clears T in exactly $2t + 1$ source steps and does not change H or X.
3. The copy-cache code begins at 4700. Its H loop is skipped because $H = 0$, while its scratch-restoration loop consumes the outstanding $X = 1$ and increments H to one.
4. The inherited NIP test begins at 4707 and recognizes precisely this $H = 1$. It reaches the halt primitive at ordinal 4718.

Counting transitions from 4696 to the halt primitive gives exactly

$$2t + 15.$$

The proof is a single unary-loop induction plus the eleven fixed post-clear edges. No inference about a very long simulated prefix is needed.

Both AX and MP return to 4696 with $A = H = X = 0$ according to their unchanged arithmetic contracts. In particular, the supplied 69,744-step blank trace and 172-step AX-prefix match put the exact table inside the AX begun with registers $[1, 0, 3, 0, 0, 0, 0, 0, 0]$. Completing that AX returns with gate/top zero and the required zero selector/cache/scratch. The source then reaches its halt primitive after fifteen further source transitions. Subject to the supplied complete primitive simulation, the archived exact binary table therefore halts on blank tape *unconditionally*. It cannot provide the requested nonhalting equivalence when ZFC is consistent.

The correction does not dispute the primitive runtime checker: that checker correctly implements the literal source. The failed boundary is between the literal source and the advertised high-level parser. Consequently the proposed first-field bias, candidate steps of one or two, and cofinality problem do not describe this exact machine. Those ideas may remain relevant to a repaired source that actually returns to its scheduler, but cannot calibrate these bytes.

The accompanying independent checker resolves both complete guarded control graphs, verifies the changed edge, records the exact fifteen-step suffix at $t = 0$, and checks the affine count for $0 \le t \le 31$. The finite count check corroborates the explicit unary-loop proof; it does not replace that proof or rerun the full physical certificates.

D Consecutive successful codes for an abstract biased parser

This section proves an arithmetic completeness tool for a possible future encoding. It is not a description of the archived scratch-biased table, whose source continuation is incorrect, and it is not needed to calibrate ST215, whose candidate counter advances normally by one. In particular, no additional physical state bound or implementation claim follows here.

Let P be Cantor pairing and f its right projection. Consider an abstract parser with a bit $b \in \{0, 1\}$, set to zero after S and to one after MP or AX. At a positive continuation pointer L, it decodes the first field from $L - 1 + b$ and reads all further command fields by ordinary right projection. A fresh candidate is decoded without bias. Use

$$\begin{aligned} \mathrm{fields}(\mathsf{S}) &= [0], \\ \mathrm{fields}(\mathsf{MP}(x)) &= [1, x], \\ \mathrm{fields}(\mathsf{AX}(c; x, y)) &= [c + 2, y, x]. \end{aligned}$$

The retained arithmetic store may be arbitrary and need not be determined by the pointer or bias. Assume fresh-candidate numbers increase by either one or two; the choice of increment may depend on the entire prior run.

A consecutive synchronization pair. For any $r > 0$, put

$$a = P(2, P(1, P(0, r))), \qquad z = P(1, a), \qquad n = P(1, z).$$

Fresh candidate n parses

$$\mathsf{MP}(1); \mathsf{AX}(0; 0, 1)$$

and reaches pointer r with bias one. Indeed its first command leaves pointer a and sets bias one, so the next selector is decoded from a, without an effective decrement. The three AX fields are $[2, 1, 0]$.

For every $z \geq 0$, the elementary Cantor identity is

$$P(1, z) + 1 = P(0, z + 1).$$

Consequently fresh candidate $n + 1$ begins with S, leaving pointer $z + 1$ and bias zero. Unless that S already halts, the next selector is read from $z = P(1, a)$, so it is MP with argument two and remaining pointer $P(1, P(0, r))$. Its bias is one, and the next MP therefore has argument zero and leaves pointer r. Thus the second word is

$$\mathsf{S}; \mathsf{MP}(2); \mathsf{MP}(0),$$

and it too reaches pointer r, bias one, or halts sooner.

This synchronizes only the parser position and bias. It does *not* assert equality of the two retained arithmetic stores. A common suffix must therefore begin with a reset before using a compiler whose input requires zero counter and cache.

Positive-tail word encoding. Let $F(\vec{a}, r)$ be right-nested pairing of a finite field list over residual r, and let $\beta(k)$ be the bias produced by command k. For a finite command word V and a positive unused tail q, define

$$\begin{aligned} E_b([], q) &= q, \\ E_b(k :: V, q) &= 1 - b + F(\text{fields}(k), E_{\beta(k)}(V, q)). \end{aligned}$$

Pairing dominates its right argument, so induction gives $E_b(V, q) \geq q > 0$. From this pointer and bias b, the parser subtracts $1 - b$ and obtains exactly the nested fields of k. It then reaches the suffix pointer with precisely the bias required by the induction hypothesis. Thus it executes all of V, unless a command halts earlier, and leaves q if the whole word returns.

The positivity of the unused tail is necessary. With base tail zero, one would obtain $E_1([\mathsf{S}], 0) = 0$, which the scheduler would treat as a fresh fetch; the intended final S would never execute. The positive tail avoids this zero alias without changing any command field.

Cofinal successful pairs. Suppose a strong refutation block $B_1; H$ halts from every $(t, w, 0, 0)$, as in the inherited short-transfer compiler. Let V be an AX reset followed by that block. It is successful from every retained store. For each $q \geq 1$, take $r = E_1(V, q)$ and construct the consecutive pair $n(r), n(r) + 1$ above. The first synchronization word reaches V from some retained store; the second either halts or does the same. Therefore both codes are successful from every initial retained store. Moreover

$$n(r) \geq r \geq q,$$

so these successful consecutive pairs are cofinal.

Every strictly increasing integer sequence with gaps at most two meets every sufficiently late consecutive pair. Otherwise, on first crossing n, it would jump from a value at most $n - 1$ to a value at least $n + 2$, a gap of at least three. This argument does not assume that the gap choices are independent of the candidate, parser bias or arithmetic data. Thus skipping at most one candidate cannot defeat the successful pairs.

Each nonhalting candidate also drains. For $L > 0$, the first unpairing input $v = L - 1 + b$ satisfies $0 \leq v \leq L$. If $v = 0$, its right projection is zero and is less than L. If $v > 0$, then $f(v) < v \leq L$; equality $f(v) = v$ would force the triangular part of P to vanish and hence $v = 0$. Further command fields only decrease the residual. Every step therefore strictly decreases the positive continuation pointer. The scheduler reaches each subsequent selected candidate in finitely many commands unless it has already halted.

These are all-natural arithmetic arguments. The companion checker `check_biased_sync.py` corroborates the two parsed prefixes for 1,000 positive residuals and checks 18 positive-tail word encodings. Those finite checks do not establish an implementation, protected-tail soundness, or a physical state count. An implementation using this lemma would still need those separate obligations.

E Complete dense transition table

Start is 0 on the all-zero tape, and halt is 215. Each entry is *write, direction, target*. There is no row for halt. The numbering is the sorted native working-state numbering, starting at zero. The full bijection is `machine/canonicalization.json`.

q	read 0	read 1	q	read 0	read 1
0	1,R,91	1,R,0	108	0,R,148	1,R,183
1	0,L,1	1,L,2	109	0,L,204	0,L,94
2	0,L,3	1,L,2	110	0,R,123	1,R,103
3	0,L,5	1,L,2	111	0,R,206	1,R,180
4	1,R,6	0,L,4	112	0,R,162	1,R,178
5	0,L,9	1,L,2	113	0,R,146	1,R,144
6	0,R,6	0,R,10	114	0,R,157	1,R,197
7	0,R,83	1,R,187	115	0,L,130	0,L,1
8	0,R,14	1,R,13	116	0,R,38	1,R,72
9	0,L,15	1,L,2	117	0,R,61	1,R,131
10	0,L,16	0,R,10	118	0,R,61	1,R,132
11	0,L,26	1,R,128	119	0,R,70	0,L,16
12	0,R,19	1,R,20	120	0,R,14	1,R,131
13	0,L,88	1,L,31	121	0,R,21	1,R,133
14	0,L,11	0,R,22	122	0,R,145	1,R,175
15	0,L,23	1,L,2	123	0,R,135	1,L,20
16	1,L,1	0,L,1	124	0,R,129	1,R,177
17	0,R,24	0,R,32	125	0,R,201	1,R,168
18	0,R,198	1,R,197	126	0,R,75	1,R,173
19	0,R,28	1,R,29	127	0,R,17	1,R,212
20	0,R,30	1,R,31	128	0,R,75	1,R,134
21	0,R,24	0,R,59	129	0,R,12	1,R,139
22	0,R,33	1,R,22	130	0,R,30	1,R,69
23	0,L,34	1,L,2	131	0,R,38	1,L,123
24	0,L,35	0,R,0	132	0,R,151	0,L,16
25	0,R,36	1,R,25	133	0,R,152	0,L,16
26	0,R,127	1,R,37	134	0,R,158	1,R,137
27	0,R,38	0,L,39	135	0,R,24	0,R,47
28	0,R,40	1,R,41	136	0,R,138	1,R,177
29	0,R,42	1,R,43	137	0,R,191	1,R,93
30	0,R,44	1,R,45	138	0,R,95	1,R,156
31	0,R,21	1,R,46	139	0,R,172	1,R,163
32	0,R,47	1,R,32	140	0,R,12	1,R,156
33	0,R,32	1,R,33	141	0,R,87	1,L,163
34	0,L,49	1,L,2	142	0,R,87	1,R,155
35	1,R,35	1,L,1	143	0,L,150	1,L,160
36	0,R,50	1,R,36	144	0,R,83	1,R,87
37	0,R,13	1,R,52	145	0,R,156	1,R,12
38	0,L,11	0,R,36	146	0,R,82	1,R,141
39	0,L,4	1,L,53	147	0,R,161	1,R,165
40	0,R,24	0,R,25	148	0,L,66	1,L,139
41	0,R,13	0,L,16	149	0,R,146	1,R,199
42	0,R,55	1,R,56	150	0,R,156	1,R,113
43	0,R,57	1,R,58	151	0,R,96	1,R,96
44	0,R,24	0,R,54	152	0,R,48	0,R,32
45	0,R,60	1,R,61	153	0,R,68	1,R,200
46	0,R,62	0,L,16	154	0,L,207	1,R,95
47	0,R,25	1,R,47	155	0,R,165	1,R,176
48	0,L,35	0,R,63	156	0,R,19	1,R,167
49	0,L,74	1,L,2	157	0,R,170	1,R,162
50	0,R,16	1,R,50	158	0,R,156	1,R,95

q	read 0	read 1	q	read 0	read 1
51	0,R,145	1,R,141	159	0,R,124	1,R,161
52	0,R,67	1,R,68	160	0,R,96	1,R,159
53	0,L,39	1,L,53	161	0,L,7	1,R,140
54	0,R,22	1,R,54	162	0,L,108	1,R,170
55	0,L,11	0,R,50	163	0,R,144	1,R,169
56	0,R,60	0,L,16	164	0,R,193	1,R,28
57	0,R,69	1,R,70	165	0,R,158	1,R,141
58	0,R,61	1,R,71	166	0,R,175	1,R,175
59	0,R,54	1,R,59	167	0,R,178	1,R,121
60	0,R,48	0,R,25	168	0,R,143	1,R,162
61	0,L,28	1,L,72	169	0,R,75	1,R,60
62	0,L,116	1,L,30	170	0,R,143	1,R,87
63	0,R,73	1,R,63	171	0,R,103	1,R,95
64	0,R,96	1,R,148	172	0,R,138	1,R,83
65	1,R,181	1,L,53	173	0,R,172	1,R,75
66	0,R,142	1,R,143	174	0,R,20	1,R,37
67	0,R,76	1,R,215	175	0,R,140	1,L,147
68	0,L,193	1,R,31	176	0,R,96	1,R,51
69	0,R,40	1,R,62	177	0,R,165	1,R,51
70	0,L,88	1,R,77	178	0,R,17	1,L,130
71	0,R,55	1,L,78	179	0,R,76	1,R,115
72	0,R,79	1,R,80	180	0,R,105	1,R,123
73	1,R,0	1,R,63	181	0,L,39	1,L,109
74	0,R,74	1,R,81	182	0,R,168	1,R,156
75	0,L,143	1,L,139	183	0,R,148	1,R,114
76	0,R,84	1,R,85	184	0,R,11	1,R,97
77	0,R,88	1,L,131	185	0,R,189	1,R,98
78	0,L,107	1,R,56	186	0,R,144	1,R,195
79	0,R,48	0,R,33	187	0,R,154	1,R,188
80	0,R,90	0,L,41	188	0,R,172	1,R,182
81	0,R,194	1,R,174	189	0,R,64	1,R,7
82	0,R,95	1,R,12	190	0,R,148	1,R,126
83	0,R,87	1,R,96	191	0,R,153	1,R,111
84	0,R,203	1,R,21	192	0,R,150	1,R,122
85	0,R,115	1,R,99	193	0,R,89	1,R,87
86	0,R,148	1,R,112	194	0,R,214	1,R,115
87	0,R,101	1,R,102	195	0,R,202	1,R,108
88	0,R,79	1,R,78	196	0,R,148	1,R,166
89	0,R,100	1,R,171	197	0,R,196	1,R,157
90	0,R,104	1,R,105	198	0,R,138	1,R,136
91	0,R,65	0,R,0	199	0,R,192	1,R,18
92	0,R,27	1,R,106	200	0,R,87	1,R,205
93	0,R,179	0,L,204	201	0,R,160	1,R,190
94	1,L,109	1,L,94	202	0,R,125	1,R,149
95	0,R,19	1,R,110	203	0,R,44	1,R,99
96	0,R,60	1,R,60	204	0,L,4	0,L,204
97	0,R,164	1,R,154	205	0,R,193	1,R,138
98	0,R,66	1,R,186	206	0,R,161	1,R,207
99	0,R,104	0,L,204	207	0,R,208	1,R,179
100	0,R,116	1,R,117	208	0,R,141	1,R,209
101	0,R,71	1,R,58	209	0,R,142	1,R,210
102	0,R,57	1,R,118	210	0,R,211	1,R,86
103	0,R,21	1,R,119	211	0,R,185	1,R,162
104	0,R,21	0,L,10	212	0,R,184	1,L,72
105	0,R,61	1,R,41	213	0,R,92	1,R,107
106	0,R,120	1,R,8	214	0,R,213	1,R,127
107	0,R,100	1,R,121			